Detailed Technical Report

A Novel Strategy for Minimum Attribute Reduction Based on Rough Set Theory and Fish Swarm Algorithm

---

A Comprehensive Mathematical and Algorithmic Analysis

Submitted by Abhishek
22mcce22

Original Paper by:
Yuebin Su & Jin Guo
Computational Intelligence and Neuroscience, 2017
Report Prepared As:
In-Depth Mathematical & Algorithmic Review

April 2026

Contents

1 Introduction and Motivation **3**
1.1 The Problem of High-Dimensional Data . . . 3
1.2 Why Minimum Attribute Reduction? . . . 3
1.3 Paper Contribution . . . 3

2 Rough Set Theory: Mathematical Foundations **4**
2.1 Information System and Decision Table . . . 4
2.2 Indiscernibility Relation . . . 4
2.3 Rough Approximations . . . 4
2.4 Attribute Dependency (Approximation Quality) . . . 5
2.5 Core Attributes . . . 5
2.6 Attribute Reduct . . . 6

3 Fish Swarm Algorithm: Principles and Mathematics **6**
3.1 Biological Inspiration . . . 6
3.2 Mathematical Representation of an Artificial Fish . . . 6
3.3 The Three Behavioural Equations . . . 7
3.3.1 Preying Behaviour . . . 7
3.3.2 Swarming Behaviour . . . 7
3.3.3 Following Behaviour . . . 8
3.4 Adaptive Parameters . . . 8

4 A Novel Coding Method for Attribute Subsets **9**
4.1 The Core Challenge: Encoding Combinations as Integers . . . 9
4.2 Lexicographic Ordering of Combinations . . . 9
4.3 The Counting Matrix $M(T(i,j))$ . . . 9
4.4 The Decoding Bijection h . . . 10
4.5 Why This Encoding is Powerful . . . 11

5 The RFSA-RST Algorithm **12**
5.1 Encoding in the FSA Context . . . 12
5.2 Fitness Function . . . 12
5.3 Full Algorithm: RFSA-RST . . . 13
5.4 Key Design Decisions Explained . . . 13

6 Experimental Evaluation **14**
6.1 Datasets . . . 14
6.2 Competing Algorithms . . . 14

6.3 Performance Metrics 14
6.4 Results: Reduct Quality 15
6.5 Results: Success and Timing 15
6.6 Time Efficiency Analysis: Loop-by-Loop Comparison 16

7 Synthesis and Critical Analysis 16
7.1 Strengths of the Proposed Method 16
7.2 Limitations and Future Work 17

8 Complete Notation Reference 18

9 Conclusion 18

1 Introduction and Motivation

1.1 The Problem of High-Dimensional Data

In modern data mining, datasets routinely contain tens or hundreds of attributes (features). Many of these attributes are *redundant* — they do not contribute new discriminative information beyond what other attributes already provide. Including such redundant attributes leads to:

- **Increased computational cost:** More attributes mean larger search spaces and longer processing times.
- **Degraded model performance:** Redundant features can introduce noise, leading to overfitting.
- **Reduced interpretability:** Models built on fewer, meaningful features are easier to understand.

Attribute Reduction (AR) is a formal preprocessing step that removes redundant or insignificant attributes while preserving the *classification ability* of the original dataset. In other words, after reduction, the system should still be able to distinguish between all objects that were distinguishable before reduction.

1.2 Why Minimum Attribute Reduction?

Among all possible valid reductions (called *reducts*), the **minimum reduct** is the one with the smallest number of attributes — the most compact sufficient description. Finding such a minimum reduct is known to be **NP-hard**, meaning no known polynomial-time exact algorithm exists. This motivates the use of *heuristic* and *metaheuristic* methods.

1.3 Paper Contribution

The paper by Su and Guo (2017) proposes **RFSA-RST** (Rough set & Fish Swarm Algorithm for Attribute Reduction), which combines:

1. **Rough Set Theory (RST):** Provides a rigorous mathematical framework for defining attribute dependency and core attributes.
2. **Fish Swarm Algorithm (FSA):** A biologically-inspired metaheuristic for global optimization.
3. **A novel integer encoding/decoding scheme:** Maps each combination (subset) of attributes to a unique integer, enabling FSA to efficiently explore fixed-cardinality subsets.

2 Rough Set Theory: Mathematical Foundations

2.1 Information System and Decision Table

A **decision table** is a formal representation of a dataset:

$$S = \{U,\ A,\ V,\ f\}$$

where:

- $U = \{x_1, x_2, \ldots, x_n\}$ is a **non-empty finite set of objects** (instances/rows in the dataset).
- $A = C \cup D$ is the **attribute set**, partitioned into:
  - C = set of **condition attributes** (input features),
  - D = set of **decision attributes** (class labels/output).
- $V = \bigcup_{a \in A} V_a$ where V_a is the **domain (range of values)** of attribute a.
- $f : U \times A \to V$ is an **information function** that assigns a value $f(x, a) \in V_a$ to each object x for each attribute a.

Example 2.1. *Consider a medical dataset where* $U = \{patient_1,\ patient_2, \ldots\}$, $C = \{age,$ *blood pressure, cholesterol*$\}$, $D = \{disease\}$. *Then* $f(patient_1, age) = 45$ *assigns the value 45 to* $patient_1$*'s age attribute.*

2.2 Indiscernibility Relation

Given any subset $R \subseteq C \cup D$, the **indiscernibility relation** $\mathrm{IND}(R)$ groups together objects that are identical on all attributes in R:

$$\mathrm{IND}(R) = \{(x, y) \mid f(x, a) = f(y, a),\ \forall\, a \in R,\ x, y \in U\} \tag{1}$$

Interpretation: Two objects x and y are *indiscernible* with respect to R if they agree on every attribute in R. This is an **equivalence relation** (reflexive, symmetric, transitive), partitioning U into *equivalence classes* denoted $[x]_R$ — the class of all objects indiscernible from x.

The **quotient set** $U/\mathrm{IND}(R)$ is the collection of all such equivalence classes, representing the granularity of the information provided by the attributes in R.

2.3 Rough Approximations

For a concept (set) $X \subseteq U$ and attribute set R, two approximations are defined:

Definition 2.1 (**Lower Approximation**).

$$\underline{R}X = \{x \in U \mid [x]_R \subseteq X\}$$

The lower approximation contains all objects that can be certainly classified as belonging to X using attributes R. If an object's entire equivalence class falls within X, we are certain it belongs to X.

Definition 2.2 (**Upper Approximation**).

$$\overline{R}X = \{x \in U \mid [x]_R \cap X \neq \emptyset\}$$

The upper approximation contains all objects that possibly belong to X.

Definition 2.3 (**Positive Region**). *The R-positive region of the decision D is:*

$$\mathrm{POS}_R(D) = \bigcup_{X \in U/\mathrm{IND}(D)} \underline{R}X$$

This is the set of all objects in U that can be unambiguously classified into one of the decision classes using the attributes in R. It represents how much of the universe can be positively determined by R.

2.4 Attribute Dependency (Approximation Quality)

The **approximation quality** γ_R measures the proportion of objects in U that can be correctly classified using attribute set R:

$$\gamma_R = \frac{|\mathrm{POS}_R(D)|}{|U|} \tag{2}$$

Notation breakdown:

- $|\mathrm{POS}_R(D)|$: the number of objects in the positive region.
- $|U|$: total number of objects.
- $\gamma_R \in [0, 1]$: value 1 means perfect classification with R; value 0 means R tells us nothing about D.

Key property: $\gamma_C = 1$ if and only if C perfectly determines D (no ambiguity in the dataset). In practice, γ_C may be less than 1 for noisy datasets.

2.5 Core Attributes

The **core** of condition attribute set C is defined as:

$$\mathrm{Core} = \{a \in C \mid \gamma_{C \setminus \{a\}} \neq \gamma_C\} \tag{3}$$

Interpretation: An attribute a belongs to the core if and only if *removing a from C decreases the approximation quality.* Core attributes are **indispensable** — every valid reduct must contain all core attributes.

Remark 2.1. *The core can be computed using the **discernibility matrix**: a matrix M where entry $M(x_i, x_j)$ contains the set of attributes that distinguish objects x_i and x_j. An attribute a is in the core if and only if it appears alone in some entry of M.*

2.6 Attribute Reduct

Definition 2.4 (**Reduct**). *A subset $R \subseteq C$ is a **reduct** if:*

1. $\gamma_R = \gamma_C$ *(classification power is preserved), and*
2. $\forall\, a \in R : \gamma_{R \setminus \{a\}} \neq \gamma_R$ *(no attribute in R is redundant).*

Definition 2.5 (**Minimum Reduct**). *A reduct R^* is a **minimum reduct** if $|R^*| \leq |R|$ for all reducts R, i.e., it has the smallest cardinality among all valid reducts.*

Finding the minimum reduct is an **NP-hard combinatorial optimisation problem** since one must search exponentially many subsets of C.

3 Fish Swarm Algorithm: Principles and Mathematics

3.1 Biological Inspiration

The Fish Swarm Algorithm (FSA) simulates the collective foraging behaviour of fish schools. Individual fish (artificial fish, AF) search for food-rich regions through three primary behaviours:

1. **Random/Preying:** A fish explores randomly or moves toward a better-fed neighbour.
2. **Swarming:** A fish moves toward the centre of its neighbourhood if that centre is food-rich.
3. **Following:** A fish follows the best-performing neighbour in its visual field.

A **bulletin board** records the global best fitness found so far, enabling information sharing across the entire population.

3.2 Mathematical Representation of an Artificial Fish

Each artificial fish i is represented as a D-dimensional position vector:

$$X_i = (x_{i1},\ x_{i2},\ \ldots,\ x_{iD})$$

- $X_{i|\text{next}}$: the updated position of fish i after one movement.
- $Y_i = F(X_i)$: the **fitness value** (food satisfaction) at position X_i.
- $d_{ij} = \|X_i - X_j\|$: **Euclidean distance** between fish i and fish j, measuring their spatial proximity in the search space.

Key FSA Parameters:

- N: population size (total number of artificial fish).
- `step`: maximum movement step length.
- `visual`: the visual radius within which a fish can perceive neighbours.
- rand $\in [0, 1]$: a random scalar drawn uniformly.
- σ: crowd factor (prevents fish from over-crowding a region).

3.3 The Three Behavioural Equations

3.3.1 Preying Behaviour

Preying is the most fundamental behaviour. Fish i randomly selects another fish j within its visual range. If j has higher fitness, fish i moves a fraction of `step` in the direction of j:

$$X_{i|\text{next}} = \begin{cases} X_i + \text{rand} \times \text{step} \times \dfrac{X_j - X_i}{\|X_j - X_i\|} & \text{if } Y_i < Y_j \\ X_i + \text{rand} \times \text{step} & \text{else (random move)} \end{cases} \tag{4}$$

Mathematical meaning:

- $\dfrac{X_j - X_i}{\|X_j - X_i\|}$ is the **unit direction vector** pointing from X_i toward X_j.
- Multiplying by rand$\times$step ensures the fish moves a random fraction of the maximum step toward the better position.
- If no better fish is found after several trials, the fish moves randomly within its visual scope, **escaping local optima**.

3.3.2 Swarming Behaviour

Let n_f be the number of fish within the visual range of fish i, and X_i^c be the **centroid** of those neighbours:

$$X_{i|\text{next}} = \begin{cases} X_i + \text{rand} \times \text{step} \times \dfrac{X_i^c - X_i}{\|X_i^c - X_i\|} & \text{if } \sigma Y_i < \dfrac{Y^c}{n_f} \\ \text{prey}(X_i) & \text{else} \end{cases} \tag{5}$$

Mathematical meaning:

- The condition $\sigma Y_i < \dfrac{Y^c}{n_f}$ checks two things simultaneously:

1. The neighbourhood centre has *higher per-fish fitness*: $Y^c/n_f > Y_i$ (after scaling by σ).
2. The centre is *not too crowded* ($\sigma > 1$ makes the condition harder to satisfy when n_f is large).

- $\dfrac{X_i^c - X_i}{\|X_i^c - X_i\|}$ is the unit vector toward the swarm centre.
- If the condition is not met, the fish falls back to **preying behaviour**.

3.3.3 Following Behaviour

Let $X_i^{\max}$ be the fish with the **greatest fitness** in the visual scope of fish i, with fitness $Y^{\max}$:

$$X_{i|\text{next}} = \begin{cases} X_i + \text{rand} \times \text{step} \times \dfrac{X_i^{\max} - X_i}{\|X_i^{\max} - X_i\|} & \text{if } \sigma Y_i < \dfrac{Y^{\max}}{n_f} \\ \text{prey}(X_i) & \text{else} \end{cases} \tag{6}$$

Mathematical meaning:

- Fish i *follows the local best* if that best is significantly better and the region isn't overcrowded.
- This implements **exploitation**: once a promising region is found, fish converge toward the best individual there.
- Equations (4)–(6) together balance **exploration** (preying, random moves) and **exploitation** (swarming, following).

3.4 Adaptive Parameters

The `visual` and `step` parameters dynamically adapt over the course of the algorithm:

$$\text{visual}_{t+1} = \text{visual}_t \times f\Big(4 \times \frac{\text{gen}}{\text{GEN}},\ 0,\ 2\Big) \tag{7}$$

$$\text{step}_{t+1} = \text{step}_t \times g\Big(\frac{\text{gen}}{\text{GEN}}\Big) \tag{8}$$

where:

- gen: current generation number; GEN: total number of generations.
- $f(\cdot)$: the **Lorentzian function**, which has a bell-shaped profile — large early on (broad exploration), narrowing later (fine-grained search).
- $g(\cdot)$: the **normal distribution function**, similarly modulating step size from large to small.

Intuition: Early in the search, fish should take large steps and look far (broad exploration). As the algorithm progresses, smaller steps and shorter visual range lead to refined local search near promising solutions.

4 A Novel Coding Method for Attribute Subsets

4.1 The Core Challenge: Encoding Combinations as Integers

Attribute reduction requires searching over *subsets* of the attribute set $C = \{1, 2, \ldots, n\}$. Standard FSA operates on continuous or integer vectors, not on sets. The paper proposes an elegant bijection between integers and fixed-size subsets (combinations), enabling FSA to operate directly on integers while each integer implicitly represents an attribute subset.

4.2 Lexicographic Ordering of Combinations

Let $C(n, m) = \{\alpha \mid \alpha \subseteq C,\ |\alpha| = m\}$ be all subsets of C of size m. The cardinality is $|C(n, m)| = \binom{n}{m}$.

Every element $\alpha = (a_1, a_2, \ldots, a_m) \in C(n, m)$ is sorted in increasing order: $a_1 < a_2 < \cdots < a_m$. The set $C(n, m)$ is then arranged in **lexicographic order**: α precedes $\beta = (b_1, \ldots, b_m)$ if and only if there exists k such that $a_i = b_i$ for all $i < k$ and $a_k < b_k$.

Proposition 4.1. *For $\alpha = (a_1, \ldots, a_m) \in C(n, m)$, the i-th element satisfies:*

$$i \leq a_i \leq n + 1 - m, \quad \forall\, i \in \{1, \ldots, m\}$$

Proof sketch: *The lower bound $i \leq a_i$ follows because there must be at least $i-1$ elements smaller than a_i in α (namely $a_1, \ldots, a_{i-1}$), so $a_i \geq i$. The upper bound $a_i \leq n + 1 - m$ ensures there are enough elements $\{a_i + 1, \ldots, n\}$ to fill the remaining $m - i$ positions after position i.*

Example 4.1 (Table $C(7, 4)$)**.** *For $n = 7, m = 4$, there are $\binom{7}{4} = 35$ combinations. The first few in lexicographic order are:*

$$1 : (1234), \quad 2 : (1235), \quad 3 : (1236), \quad \ldots, \quad 22 : (2346), \quad \ldots, \quad 35 : (4567)$$

4.3 The Counting Matrix $M(T(i, j))$

To establish a bijection between $\{1, 2, \ldots, \binom{n}{m}\}$ and $C(n, m)$, the paper constructs a matrix M of size $m \times n$ where:

$$T(i, j) = \begin{cases} \binom{n-j}{m-i} & 1 \leq i \leq m,\ i \leq j \leq n + 1 - m \\ 0 & \text{otherwise} \end{cases} \tag{9}$$

Interpretation of $T(i, j)$: If the i-th chosen element is j (i.e., $a_i = j$), then there are $\binom{n-j}{m-i}$ ways to choose the remaining $m - i$ elements from $\{j + 1, j + 2, \ldots, n\}$. Thus $T(i, j)$ counts how many combinations have $a_i = j$.

A crucial recursive identity follows directly:

$$T(i,j) = \sum_{k=j+1}^{n} T(i+1,k), \quad \text{if } T(i,j) \neq 0 \tag{10}$$

Proof: The combinations counted by $T(i,j)$ (where $a_i = j$) are exactly those where a_{i+1} takes any value from $\{j+1, \ldots, n+1-m+1\}$, giving $\sum_{k=j+1}^{n} T(i+1,k)$.

Example 4.2 (Matrix for $C(7,4)$)**.**

$$M = \begin{pmatrix} 20 & 10 & 4 & 1 & 0 & 0 & 0 \\ 0 & 10 & 6 & 3 & 1 & 0 & 0 \\ 0 & 0 & 4 & 3 & 2 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Row i gives counts for the i-th chosen element. For example, $T(1,1) = \binom{7-1}{4-1} = \binom{6}{3} =$ 20 means there are 20 combinations where the first element is 1.

4.4 The Decoding Bijection h

The bijection $h : \{1, 2, \ldots, \binom{n}{m}\} \to C(n,m)$ is defined via Algorithm 1:

Algorithm 1 DetoCo(x, n, m) — Decode integer x to combination

Require: Integer $x \leq \binom{n}{m}$, parameters n (total attributes), m (subset size)
Ensure: Combination $\alpha = (a_1, a_2, \ldots, a_m)$

```
Compute matrix M = (T(i,j))_{m×n} via Equation (9)
l ← 1
Find smallest j_1 such that Σ_{k=1}^{j_1} T(1,k) ≥ x
a_1 ← j_1;   x ← x − Σ_{k=1}^{j_1−1} T(1,k)
for l = 2 to m do
    Find smallest j_l ≥ j_{l−1} + 1 such that Σ_{k=j_{l−1}+1}^{j_l} T(l,k) ≥ x
    a_l ← j_l;   x ← x − Σ_{k=j_{l−1}+1}^{j_l−1} T(l,k)
end for
return α = (a_1, a_2, ..., a_m)
```

Algorithm logic step by step:

1. We want to find which combination is the x-th in lexicographic order.
2. For a_1: we count how many combinations start with 1, then how many start with 2, etc., until the cumulative count reaches x. The value at which this threshold is crossed gives a_1.
3. We then subtract the count of all combinations that precede those starting with a_1, re-setting x to a local rank within the sub-problem.
4. Repeat for $a_2, a_3, \ldots, a_m$, each time restricting to values $> a_{l-1}$.

Example 4.3 (Decoding $x = 22$ from $C(7,4)$)**.** *We want* $h(22) = (a_1, a_2, a_3, a_4)$ *from* $C(7,4)$.

Step 1 — Find a_1***:***

$$T(1,1) = 20 < x = 22 \leq T(1,1) + T(1,2) = 20 + 10 = 30$$
$$\Rightarrow\ a_1 = 2, \quad x \leftarrow 22 - 20 = 2$$

Step 2 — Find a_2 ***(must be*** > 2***):***

$$x = 2 < T(2,3) = 6 \quad \Rightarrow\ a_2 = 3, \quad x \leftarrow 2 - 0 = 2$$

Step 3 — Find a_3 ***(must be*** > 3***):***

$$x = 2 < T(3,4) = 3 \quad \Rightarrow\ a_3 = 4, \quad x \leftarrow 2 - 0 = 2$$

Step 4 — Find a_4 ***(must be*** > 4***):***

$$T(4,5) = 1 < x = 2 \leq T(4,5) + T(4,6) = 1 + 1 = 2$$
$$\Rightarrow\ a_4 = 6$$

Result: $h(22) = (2,3,4,6)$*, which matches the 22nd entry in Table* $C(7,4)$*.* □

4.5 Why This Encoding is Powerful

The integer encoding achieves several important goals:

1. **Fixed cardinality:** Every integer $x \in \{1, \ldots, \binom{n}{m}\}$ maps to a subset of exactly m attributes. The search space at each iteration is constrained to subsets of a fixed size.

2. **Completeness:** The mapping h is a bijection — every subset of size m appears exactly once, so the FSA explores the entire space $C(n,m)$ without duplication.

3. **Dimensionality reduction:** Instead of searching a n-bit binary vector (standard encoding), the search space is $\{1, \ldots, \binom{n}{m}\}$ — a single integer. This dramatically reduces the dimensionality of the FSA search problem.

4. **Incremental strategy:** The outer loop increases m by 1 each iteration, starting from the core size. This ensures minimality: the algorithm returns the first m for which a valid reduct exists.

5 The RFSA-RST Algorithm

5.1 Encoding in the FSA Context

In the RFSA-RST context:

- $C = \{1, 2, \ldots, n\}$ is the set of **non-core** condition attributes (after removing the core).
- Each artificial fish i holds a position $r_i \in \{1, \ldots, \binom{n}{m}\}$ — a single integer.
- $h(r_i)$ via Algorithm 1 decodes this to an attribute subset of size m.
- The actual candidate reduct is $h(r_i) \cup \text{Core}$.

Since the FSA position update (Equations (4)–(6)) may produce non-integer or out-of-bound values, a corrected update rule is used:

$$X_{i|\text{next}} = \begin{cases} X_i + \text{round}(\Delta) + 1 & \text{if } \Delta > 0 \\ X_i + \text{round}(\Delta) - 1 & \text{else} \end{cases} \tag{11}$$

where Δ is the real-valued increment computed by the standard FSA equations. The ± 1 correction ensures the fish always moves in a meaningful direction (avoiding stagnation at the same integer value).

5.2 Fitness Function

For a fish at integer position X, the fitness is:

$$F(X) = \gamma_{h(X) \cup \text{Core}} \tag{12}$$

Meaning:

- Decode X to subset $h(X)$ using Algorithm 1.
- Combine with the core: $P = h(X) \cup \text{Core}$.
- Compute the approximation quality γ_P via Equation (2).
- A fitness value of γ_C (maximum possible) means P is a valid reduct.

Why this fitness works: It is a simple, single scalar that quantifies how well the candidate attribute subset classifies the data. No complex multi-objective formulation is needed.

5.3 Full Algorithm: RFSA-RST

Algorithm 2 RFSA-RST(DS) — Minimum Attribute Reduction via RST and FSA

Require: Decision table $DS = (U, C, D, V, f)$
Ensure: Minimum reduct set Red

```
Red ← ∅
Compute γ_C and Core using discernibility matrix
Compute γ_Core
if γ_Core = γ_C then
    Red ← Core; return Red                          ▷ Core alone is sufficient
end if
L ← C \ Core;   n ← |L|;   m ← 0
repeat                                              ▷ Outer loop: increase subset size
    m ← m + 1;   avg_old ← 0
    Randomly initialise population: {r_1, r_2, ..., r_N} ⊆ {1, ..., (n choose m)}
    repeat                                          ▷ Inner FSA loop
        sum ← 0
        for i = 1 to N do
            P_i ← h(r_i) ∪ Core                     ▷ Decode integer to subset, add Core
            Compute F(P_i) = γ_{P_i} via Equation (12)
            if F(P_i) = γ_C then
                Red ← Red ∪ {P_i}                   ▷ Found a valid reduct!
            else
                sum ← sum + F(P_i)
            end if
        end for
        avg_new ← sum/N
        change ← |avg_new − avg_old|;   avg_old ← avg_new
        Execute swarming, following, and preying behaviours for all fish
    until change ≤ 0.001                            ▷ Convergence criterion
until Red ≠ ∅
return Red
```

5.4 Key Design Decisions Explained

1. **Core identification first:** By pre-computing the core and only searching over non-core attributes, the search space is dramatically reduced. The core is guaranteed to be in every reduct, so it need not be explored.

2. **Incremental cardinality (m loop):** Starting at $m = 1$ and increasing until a reduct is found guarantees that the *first* successful m gives the minimum reduct cardinality. This is key to finding minimum (not just any) reducts.

3. **Convergence criterion (change ≤ 0.001):** If the average fitness of the population changes by less than 0.001 between two successive generations, the FSA has stagnated at this cardinality level and the outer loop increments m.

4. **Population diversity:** The random initialisation at each outer loop iteration ensures fresh exploration for each cardinality level.

6 Experimental Evaluation

6.1 Datasets

Six benchmark datasets from the UCI Machine Learning Repository were used:

Table 1: Benchmark datasets used for evaluation

Dataset	**Objects (n)**	**Condition Attributes (m)**
Lung-cancer	26	52
Soybean-small	47	36
Sponge	76	45
Zoo	101	17
Blance	625	5
Spect	187	23

The datasets span a range of sizes and difficulties: from small ($n = 26$, high-dimensional $m = 52$) to larger ($n = 625$, low-dimensional $m = 5$).

6.2 Competing Algorithms

- **PSO-RST** (Wang et al., 2007): Particle Swarm Optimisation combined with RST.
- **RGA-RST** (Wroblewski, 1995): Genetic Algorithm combined with RST.
- **RFSA-RST** (proposed): Fish Swarm Algorithm combined with RST (this paper).

All three were run 50 independent times per dataset, with identical parameter settings for fairness.

6.3 Performance Metrics

- **STL (Shortest Length):** Minimum number of attributes in the output reduct across all 50 runs. A lower value indicates the algorithm found a more compact reduct.
- **AVL (Average Length):** Mean number of attributes in the output across all 50 runs. Lower values with the same STL indicate greater *consistency* in finding minimum reducts.
- s_1 **(Success ratio):** Proportion of runs that found the minimum reduct (s_1 = runs with length equal to STL / 50).
- s_2 **(Normal ratio):** Proportion of runs that found any valid reduct (not necessarily minimum).

- **AVT (Average Running Time):** Mean wall-clock time per run in seconds.

6.4 Results: Reduct Quality

Table 2: Reduct length comparison (STL = shortest length, AVL = average length)

Dataset	**PSO-RST**		**RGA**		**RFSA-RST**	
	STL	AVL	STL	AVL	STL	**AVL**
Lung-cancer	3	4.14	3	4.02	3	**3.92**
Soybean-small	2	2.29	2	**2.32**	2	2.38
Sponge	8	8.7	8	8.4	8	**8.02**
Zoo	5	5.32	5	5.05	5	**5.0**
Blance	4	4.02	4	4.0	4	**4.0**
Spect	16	16.0	16	16.0	16	**16.0**

Key observations:

- All three algorithms achieve the same **STL** (same shortest reduct length), confirming they can all find the minimum reduct.
- **RFSA-RST achieves the best or equal AVL** on 5 out of 6 datasets, indicating superior consistency in finding the minimum reduct.
- Soybean-small is the exception where RGA has slightly better AVL, possibly because the dataset has multiple minimum reducts and FSA's constrained search occasionally misses some.

6.5 Results: Success and Timing

Table 3: Success/normal ratios and average running times

Dataset	**PSO-RST**		**RGA**		**RFSA-RST**	
	s_1/s_2	AVT(s)	s_1/s_2	AVT(s)	s_1/s_2	AVT(s)
Lung-cancer	0.06/1	0.207	0.08/1	0.810	**0.22**/1	0.226
Soybean-small	0.70/1	0.712	**0.78**/1	1.018	0.72/1	0.716
Sponge	0.60/0.92	6.416	0.62/0.98	16.113	**0.78**/0.98	6.809
Zoo	0.94/1	1.389	0.96/1	2.165	**1.0**/1	1.458
Blance	0.98/1	0.573	1.0/1	1.994	1.0/1	0.605
Spect	1.0/1	12.037	1.0/1	36.094	1.0/1	12.901

Key observations:

- **RFSA-RST achieves the best or equal s_1 (success rate)** on 5 out of 6 datasets, demonstrating superior reliability in finding minimum reducts.
- **RGA is considerably slower** (e.g., 36.09s vs 12.90s on Spect, 16.11s vs 6.81s on Sponge) because it works on variable-length binary chromosome representations.
- **RFSA-RST is slightly slower than PSO-RST** but the difference is small (e.g., 12.90s vs 12.04s on Spect), while offering substantially higher success rates.

6.6 Time Efficiency Analysis: Loop-by-Loop Comparison

A secondary experiment was conducted comparing RFSA-RST and RGA loop-by-loop (varying cardinality m from 1 to the minimum reduct length). Key finding:

- RFSA-RST's per-loop time *increases monotonically* with m because the search space $\binom{n}{m}$ grows with m.
- RGA's per-loop time is *relatively stable* because its binary encoding doesn't change dimensionality with m.
- However, RFSA-RST's total time is lower because its fixed-cardinality encoding requires *far fewer fitness evaluations per loop* to cover the relevant search space.

7 Synthesis and Critical Analysis

7.1 Strengths of the Proposed Method

1. **Guaranteed minimum cardinality:** The incremental m-loop ensures the algorithm returns the minimum-length reduct, not just any reduct. This is a key advantage over algorithms that search the full power set.
2. **Constrained but complete search:** Each FSA loop searches only over subsets of size m. This eliminates the exponential redundancy of searching subsets of mixed sizes, dramatically reducing wasted computation.
3. **Strong global optimisation via FSA:** FSA's combination of preying (random escape), swarming (collective intelligence), and following (local best exploitation) gives excellent balance between exploration and exploitation, avoiding local optima that plague greedy methods.
4. **Simple and efficient fitness evaluation:** The fitness $\gamma_{h(X)\cup\text{Core}}$ is a single dependency calculation — cheap to compute compared to wrapper-based fitness functions.
5. **Core pre-computation:** Identifying and fixing core attributes reduces the effective search space from $2^{|C|}$ to $2^{|C\setminus\text{Core}|}$, often a significant reduction.

7.2 Limitations and Future Work

1. **Slightly slower than PSO-RST:** The overhead of the DecoCo decoding algorithm (Algorithm 1) and the adaptive FSA behaviours add some computational cost compared to simpler PSO.

2. **Soybean-small anomaly:** On Soybean-small, RFSA-RST has the highest AVL (2.38 vs 2.29 for PSO-RST). This may be because Soybean-small has many valid minimum reducts and the FSA's constrained search (fixed m) occasionally explores inefficiently when many optima coexist.

3. **Starting length of core:** The paper notes as future work: confirming whether starting m from 1 (rather than from $|\text{Core}| + 1$) is optimal. In datasets where the minimum reduct is only slightly larger than the core, starting from $|\text{Core}|$ might skip unnecessary early iterations.

4. **Scalability:** For very large n (hundreds of attributes), $\binom{n}{m}$ can be astronomically large. The encoding approach may need hierarchical decomposition for extreme dimensions.

8 Complete Notation Reference

Table 4: All notation used in the paper and this report

Symbol	Meaning
U	Universe — finite set of all objects (instances)
$A = C \cup D$	Full attribute set: condition (C) + decision (D)
V	Domain of attribute values
$f(x, a)$	Value of attribute a for object x
$\mathrm{IND}(R)$	Indiscernibility relation for attribute set R
$[x]_R$	Equivalence class of object x under $\mathrm{IND}(R)$
$\underline{R}X$	Lower approximation of set X by R
$\mathrm{POS}_R(D)$	Positive region of D with respect to R
γ_R	Approximation quality (dependency) of R
Core	Set of indispensable condition attributes
$C(n, m)$	All subsets of $\{1, \ldots, n\}$ of size m
$\binom{n}{m}$	Binomial coefficient — number of m-subsets of $\{1, \ldots, n\}$
$T(i, j)$	Matrix entry: combinations with i-th element equal to j
$h(x)$	Decoding function: integer $x \mapsto$ combination in $C(n, m)$
X_i	Position of artificial fish i (an integer in RFSA-RST)
$X_{i\|\mathrm{next}}$	Updated position of fish i after one step
$Y_i = F(X_i)$	Fitness (food satisfaction) of fish i
$d_{ij} = \|X_i - X_j\|$	Euclidean distance between fish i and j
N	Population size (number of artificial fish)
`step`	Maximum step length for fish movement
`visual`	Visual radius — fish perceive neighbours within this distance
σ	Crowd factor — penalises overcrowded regions
rand	Uniform random number in $[0, 1]$
n_f	Number of fish within visual range of current fish
X_i^c	Centroid of neighbours within visual range of fish i
$X_i^{\max}$	Best-fitness neighbour within visual range of fish i
Y^c, $Y^{\max}$	Fitness at X_i^c and $X_i^{\max}$ respectively
gen, GEN	Current generation and total generations
Δ	Real-valued increment before discretisation
Red	Output: set of minimum reducts found
STL, AVL, AVT	Shortest length, average length, average time (metrics)
s_1, s_2	Success ratio, normal ratio

9 Conclusion

The paper by Su and Guo (2017) presents a principled and effective approach to the NP-hard minimum attribute reduction problem. By combining three key ideas:

1. **Rough Set Theory** for formally defining attribute dependency (γ_R), core identification, and reduct validity;
2. **A novel integer encoding** that bijects each fixed-size attribute subset to a unique integer, enabling efficient constrained search;
3. **Fish Swarm Algorithm** as the global optimiser, with strong ability to escape local optima via its three complementary behaviours;

the resulting RFSA-RST algorithm achieves superior success rates in finding minimum reducts compared to both PSO-RST and RGA, while maintaining competitive running times. The incremental cardinality strategy is mathematically elegant: it is both necessary and sufficient to guarantee minimum-cardinality output.

The work makes a meaningful contribution to the field of feature selection / attribute reduction and demonstrates how a carefully chosen encoding can transform a combinatorial set problem into a tractable integer optimisation problem amenable to population-based search.